

AtCoder Library 移植模板

自包含可粘贴版·基于 [ac-library v1.6](#)

Build Time: July 30, 2026

Contents

1	AtCoder Library (ACL) 移植	2
1.1	segtree: 单点改 + 区间查 (非递归)	2
1.2	lazy_segtree: 区间改 + 区间查 (懒标记)	4

1 AtCoder Library (ACL) 移植

本文档收录 AtCoder Library v1.6 里值得直接搬进赛场的组件，逐个改成自包含可粘贴版：

- 去掉 `#ifndef` 头尾守卫与 `namespace atcoder`，去掉 `std::` 前缀（模板里默认 `using namespace std;`）。
- 原本 `#include "atcoder/internal_bit"` 提供的 `bit_ceil / countr_zero` 内联成结构体私有 `static`，不再依赖任何 ACL 头文件。
- 模板参数统一用函数指针形式 `S (*op)(S, S)` 而非 C++17 的 `auto op`——写法与 C++14 兼容，且报错信息比 `auto + static_assert` 版本短得多。用 C++17 以上编译器时把 `S (*op)(S, S)`，`S (*e)()` 换成 `auto op`，`auto e` 也能编译，行为一致。

为什么要移植：ACL 只在 AtCoder 评测机上预装，ICPC / CCPC 赛场机没有 `atcoder/` 头文件目录，`#include <atcoder/segtree>` 直接 CE。本地开发（如 CMake 里 `include_directories(SYSTEM .../ac-library)`）可以照常用官方头，但打印稿上必须是能手抄进单文件的版本。

全文共同约定：下标 0-based，区间一律 $[l, r)$ 左闭右开——与主板子 `template-main` 各章偏好的 1-based 闭区间相反，混用时注意换算。

正确性验证：本文档代码块经 `template-check/` 下的 `acl_stress/run.sh` 与官方 ACL v1.6 及朴素暴力三方对拍（含 ASan / UBSan）通过。该脚本直接从本文档 .tex 抽 minted 块编译，改了代码不重跑就会暴露不一致。

1.1 segtree：单点改 + 区间查（非递归）

维护满足结合律的二元运算 `op` 与其单位元 `e`（即么半群 monoid）上的序列，支持单点赋值 $O(\log n)$ 、区间求积 $O(\log n)$ 、以及在线段树上二分 $O(\log n)$ 。

接口一览（ n 为序列长度，下标 0-based）：

- `segtree<S, op, e> seg(n)`：长度 n ，全部初始化为 `e()`。
- `segtree<S, op, e> seg(v)`：由 `vector<S> v` 建树， $O(n)$ 。
- `seg.set(p, x)`：令 $a_p \leftarrow x$ 。
- `seg.get(p)`：返回 a_p 。
- `seg.prod(l, r)`：返回 $op(a_l, \dots, a_{r-1})$ ； $l = r$ 时返回 `e()`。
- `seg.all_prod()`：返回全区间积， $O(1)$ 。
- `seg.max_right(l, f)`：返回最大的 r ，使 $f(op(a_l, \dots, a_{r-1}))$ 为真。要求 `f` 单调（真在前假在后）且 `f(e())` 为真。

- `seg.min_left(r, f)`：返回最小的 l ，使 $f(op(a_l, \dots, a_{r-1}))$ 为真。同样要求 `f` 单调且 `f(e())` 为真。

三个易错点：

1. `op` 与 `e` 必须是自由函数（或函数指针 / 无捕获 lambda 转成的函数指针），不能是带捕获的 lambda——它们是模板参数，编译期就要定死。
2. `e()` 必须是真单位元：求最大值时用足够小的哨兵（如 $-4e18$ ）而非 0，否则负数序列查询结果错。
3. `prod` 是 $[l, r)$ 左闭右开；从 1-based 闭区间 $[L, R]$ 换算过来是 `prod(L - 1, R)`。

```
/* AtCoder Library v1.6 atcoder/segtree
↳ 自包含移植
* 改动：去 #ifndef 守卫与 namespace atcoder,
*       去 std:: 前缀, internal_bit 的
*       bit_ceil / countr_zero 内联成私有 static
* 约定：下标 0-based, 区间 [l, r) 左闭右开
*/
template<class S, S (*op)(S, S), S (*e)()>
struct segtree {
public:
    segtree() : segtree(0) {}

    explicit segtree(int n)
        : segtree(vector<S>(n, e())) {}

    explicit segtree(const vector<S> &v)
        : _n(int(v.size())) {
        size = (int) bit_ceil((unsigned) _n);
        log = countr_zero((unsigned) size);
        d = vector<S>(2 * size, e());
        for (int i = 0; i < _n; i++)
            d[size + i] = v[i];
        for (int i = size - 1; i >= 1; i--)
            update(i);
    }

    // a[p] = x
    void set(int p, S x) {
        assert(0 <= p && p < _n);
        p += size;
        d[p] = x;
        for (int i = 1; i <= log; i++)
            update(p >> i);
    }

    // 返回 a[p]
    S get(int p) const {
        assert(0 <= p && p < _n);
        return d[p + size];
    }

    // 返回 op(a[l], ..., a[r - 1])
    S prod(int l, int r) const {
        assert(0 <= l && l <= r && r <= _n);
```

```

S sm1 = e(), smr = e();
l += size, r += size;
while (l < r) {
    if (l & 1) sm1 = op(sm1, d[l++]);
    if (r & 1) smr = op(d[--r], smr);
    l >>= 1, r >>= 1;
}
return op(sm1, smr);
}

S all_prod() const { return d[1]; }

// 最大的 r 使 f(op(a[l..r-1])) 为真
template<class F>
int max_right(int l, F f) const {
    assert(0 <= l && l <= _n);
    assert(f(e()));
    if (l == _n) return _n;
    l += size;
    S sm = e();
    do {
        while (l % 2 == 0) l >>= 1;
        if (!f(op(sm, d[l]))) {
            while (l < size) {
                l = 2 * l;
                if (f(op(sm, d[l]))) {
                    sm = op(sm, d[l]);
                    l++;
                }
            }
            return l - size;
        }
        sm = op(sm, d[l]);
        l++;
    } while ((l & -l) != 1);
    return _n;
}

// 最小的 l 使 f(op(a[l..r-1])) 为真
template<class F>
int min_left(int r, F f) const {
    assert(0 <= r && r <= _n);
    assert(f(e()));
    if (r == 0) return 0;
    r += size;
    S sm = e();
    do {
        r--;
        while (r > 1 && (r % 2)) r >>= 1;
        if (!f(op(d[r], sm))) {
            while (r < size) {
                r = 2 * r + 1;
                if (f(op(d[r], sm))) {
                    sm = op(d[r], sm);
                    r--;
                }
            }
            return r + 1 - size;
        }
        sm = op(d[r], sm);
    } while ((r & -r) != r);
    return 0;
}

```

```

private:
    int _n, size, log;
    vector<S> d;

    // 原 atcoder::internal::bit_ceil
    static unsigned bit_ceil(unsigned n) {
        unsigned x = 1;
        while (x < n) x *= 2;
        return x;
    }

    // 原 atcoder::internal::countr_zero
    static int countr_zero(unsigned n) {
        return __builtin_ctz(n);
    }

    void update(int k) {
        d[k] = op(d[2 * k], d[2 * k + 1]);
    }
};

```

用法一：区间最大值。op / e 写在全局（或匿名 namespace），然后实例化：

```

using S = ll;
S op_max(S a, S b) { return max(a, b); }
S e_max() { return -4e18; } // 单位元哨兵

// ...
int n;
cin >> n;
vector<S> a(n);
for (auto &x: a) cin >> x;
segtree<S, op_max, e_max> seg(a);

seg.set(3, 100); // a[3] = 100
cout << seg.prod(0, n) << "\n"; // 全局最大
cout << seg.prod(2, 5) << "\n"; // a[2..4]
→ 最大

```

用法二：区间和 + 线段树上二分。求「从 l 出发最长的前缀，使区间和不超过 x 」——这是 max_right 的典型用途，把「外层二分 + 内层查询」的 $O(\log^2 n)$ 压到 $O(\log n)$ ：

```

using S = ll;
S op_sum(S a, S b) { return a + b; }
S e_sum() { return 0; }

// ...
segtree<S, op_sum, e_sum> seg(a);
ll x = 10;
// r 是最大的下标，使 a[l..r-1] 之和 <= x
int r = seg.max_right(l, [&](S sum) { return sum <= x; });

```

注意 max_right 的谓词 f 可以是带捕获的 lambda（它是普通函数参数，不是模板参数），与 op / e 的限制不同。

与本模板其它线段树的取舍：ACL 版是非递归实现，常数小、代码短，但只支持单点修改；要区间修改

(区间加 / 区间赋值) 请用下一小节的 lazy_segtree, 或主板子数据结构章的递归式懒标记线段树。ACL 版另一个优势是 max_right / min_left 这对树上二分接口, 主板子的线段树模板没有等价物。

1.2 lazy_segtree: 区间改 + 区间查 (懒标记)

在 segtree 的幺半群 (S, op, e) 之上再挂一个作用 (映射) 幺半群 $(F, composition, id)$, 支持区间作用 $O(\log n)$ 、区间求积 $O(\log n)$ 、树上二分 $O(\log n)$ 。

五件套的代数约束 (写错这里就是 WA 的主要来源):

- $S \text{ op}(S \ a, S \ b)$: 结合律, $S \ e()$ 是它的单位元。
- $S \text{ mapping}(F \ f, S \ x)$: 把作用 f 施加到 x 上。注意参数顺序是「先 f 后 x 」。
- $F \text{ composition}(F \ f, F \ g)$: 复合成「先 g 再 f 」, 即数学上的 $f \circ g$, 必须满足

$$\begin{aligned} \text{mapping}(\text{composition}(f, g), x) \\ = \text{mapping}(f, \text{mapping}(g, x)). \end{aligned}$$

- $F \text{ id}()$: F 的恒等作用, $\text{mapping}(\text{id}(), x) == x$ 。
- 分配律: 作用要对 op 分配, 即

$$\begin{aligned} \text{mapping}(f, \text{op}(x, y)) \\ = \text{op}(\text{mapping}(f, x), \text{mapping}(f, y)). \end{aligned}$$

这条不成立就不能用懒标记。

最大的坑: mapping 拿不到区间长度。做「区间加 + 区间和」时, $\text{mapping}(+v, x) = x + v \cdot \text{len}$ 需要 len , 而 mapping 的入参只有 f 和 x ——所以长度必须存进 S 里, 让它随 op 一起合并。这与第 8 章那种「递归下传时天然知道 $[l, r]$ 」的写法不同, 是 ACL 版最容易踩空的地方。

接口一览 (比 segtree 多出 apply, 且因为要 push 懒标记, get / prod 不再是 const):

- $\text{seg.apply}(p, f)$: 单点作用 $a_p \leftarrow \text{mapping}(f, a_p)$ 。
- $\text{seg.apply}(l, r, f)$: 区间 $[l, r)$ 上每个元素都作用 f 。
- $\text{seg.set} / \text{get} / \text{prod} / \text{all_prod} / \text{max_right} / \text{min_left}$: 语义同 segtree。

```
/* AtCoder Library v1.6 atcoder/lazysegtree
 * 自包含移植, 改动同 segtree 小节
 * 约定: 下标 0-based, 区间 [l, r) 左闭右开
 */
template<class S, S (*op)(S, S), S (*e)(),
         class F, S (*mapping)(F, S),
         F (*composition)(F, F), F (*id)()>
struct lazy_segtree {
public:
```

```
lazy_segtree() : lazy_segtree(0) {}

explicit lazy_segtree(int n)
    : lazy_segtree(vector<S>(n, e())) {}

explicit lazy_segtree(const vector<S> &v)
    : _n(int(v.size())) {
    size = (int) bit_ceil((unsigned) _n);
    log = countr_zero((unsigned) size);
    d = vector<S>(2 * size, e());
    lz = vector<F>(size, id());
    for (int i = 0; i < _n; i++)
        d[size + i] = v[i];
    for (int i = size - 1; i >= 1; i--)
        update(i);
}

// a[p] = x
void set(int p, S x) {
    assert(0 <= p && p < _n);
    p += size;
    for (int i = log; i >= 1; i--) push(p
        ↪ >> i);
    d[p] = x;
    for (int i = 1; i <= log; i++) update(p
        ↪ >> i);
}

// 返回 a[p]
S get(int p) {
    assert(0 <= p && p < _n);
    p += size;
    for (int i = log; i >= 1; i--) push(p
        ↪ >> i);
    return d[p];
}

// 返回 op(a[l], ..., a[r - 1])
S prod(int l, int r) {
    assert(0 <= l && l <= r && r <= _n);
    if (l == r) return e();
    l += size, r += size;
    for (int i = log; i >= 1; i--) {
        if (((l >> i) << i) != 1) push(l >>
            ↪ i);
        if (((r >> i) << i) != r)
            push((r - 1) >> i);
    }
    S sml = e(), smr = e();
    while (l < r) {
        if (l & 1) sml = op(sml, d[l++]);
        if (r & 1) smr = op(d[--r], smr);
        l >>= 1, r >>= 1;
    }
    return op(sml, smr);
}

S all_prod() { return d[1]; }
```

```
// 单点作用 a[p] = mapping(f, a[p])
void apply(int p, F f) {
    assert(0 <= p && p < _n);
    p += size;
```

```

for (int i = log; i >= 1; i--) push(p
    ↪ >> i);
d[p] = mapping(f, d[p]);
for (int i = 1; i <= log; i++) update(p
    ↪ >> i);
}

```

// 区间作用, $[l, r)$ 内每个元素都施加 f

```

void apply(int l, int r, F f) {
    assert(0 <= l && l <= r && r <= _n);
    if (l == r) return;
    l += size, r += size;
    for (int i = log; i >= 1; i--) {
        if ((l >> i) << i) != l) push(l >>
            ↪ i);
        if ((r >> i) << i) != r)
            push((r - 1) >> i);
    }
    {
        int l2 = l, r2 = r;
        while (l < r) {
            if (l & 1) all_apply(l++, f);
            if (r & 1) all_apply(--r, f);
            l >>= 1, r >>= 1;
        }
        l = l2, r = r2;
    }
    for (int i = 1; i <= log; i++) {
        if ((l >> i) << i) != l) update(l
            ↪ >> i);
        if ((r >> i) << i) != r)
            update((r - 1) >> i);
    }
}

```

// 最大的 r 使 $g(\text{op}(a[l..r-1]))$ 为真

```

template<class G>
int max_right(int l, G g) {
    assert(0 <= l && l <= _n);
    assert(g(e()));
    if (l == _n) return _n;
    l += size;
    for (int i = log; i >= 1; i--) push(l
        ↪ >> i);
    S sm = e();
    do {
        while (l % 2 == 0) l >>= 1;
        if (!g(op(sm, d[l]))) {
            while (l < size) {
                push(l);
                l = 2 * l;
                if (g(op(sm, d[l]))) {
                    sm = op(sm, d[l]);
                    l++;
                }
            }
            return l - size;
        }
        sm = op(sm, d[l]);
        l++;
    } while ((l & -1) != l);
    return _n;
}

```

// 最小的 l 使 $g(\text{op}(a[l..r-1]))$ 为真

```

template<class G>
int min_left(int r, G g) {
    assert(0 <= r && r <= _n);
    assert(g(e()));
    if (r == 0) return 0;
    r += size;
    for (int i = log; i >= 1; i--)
        push((r - 1) >> i);
    S sm = e();
    do {
        r--;
        while (r > 1 && (r % 2)) r >>= 1;
        if (!g(op(d[r], sm))) {
            while (r < size) {
                push(r);
                r = 2 * r + 1;
                if (g(op(d[r], sm))) {
                    sm = op(d[r], sm);
                    r--;
                }
            }
            return r + 1 - size;
        }
        sm = op(d[r], sm);
    } while ((r & -r) != r);
    return 0;
}

```

private:

```

int _n, size, log;
vector<S> d;
vector<F> lz;

// 原 atcoder::internal::bit_ceil
static unsigned bit_ceil(unsigned n) {
    unsigned x = 1;
    while (x < n) x *= 2;
    return x;
}

// 原 atcoder::internal::countr_zero
static int countr_zero(unsigned n) {
    return __builtin_ctz(n);
}

void update(int k) {
    d[k] = op(d[2 * k], d[2 * k + 1]);
}

void all_apply(int k, F f) {
    d[k] = mapping(f, d[k]);
    if (k < size)
        lz[k] = composition(f, lz[k]);
}

void push(int k) {
    all_apply(2 * k, lz[k]);
    all_apply(2 * k + 1, lz[k]);
    lz[k] = id();
}
};

```

用法一：区间加 + 区间和。S 里带上 len 解决「mapping 不知道区间长度」：

```
struct S {
    ll sum;
    int len;
};
using F = ll; // 懒标记 = 这段区间要加的数

S op(S a, S b) {
    return {a.sum + b.sum, a.len + b.len};
}
S e() { return {0, 0}; }
S mapping(F f, S x) {
    return {x.sum + f * x.len, x.len};
}
F composition(F f, F g) { return f + g; }
F id() { return 0; }

using Seg = lazy_segtree<S, op, e, F,
    mapping, composition, id>;

// ...
int n;
cin >> n;
vector<S> a(n);
for (int i = 0; i < n; i++) {
    ll x;
    cin >> x;
    a[i] = {x, 1}; // 单点长度是 1
}
Seg seg(a);

seg.apply(1, r, 5); // [l, r) 每个元素 += 5
cout << seg.prod(1, r).sum << "\n";
```

用法二：区间赋值 + 区间最大。赋值标记要能表达「没有赋值」，用一个不可能出现的哨兵值当 id()：

```
using S = ll;
using F = ll;
const F NONE = LLONG_MIN; // 「未赋值」哨兵

S op(S a, S b) { return max(a, b); }
S e() { return -4e18; }
S mapping(F f, S x) {
    return f == NONE ? x : f;
}
// 先 g 再 f: f 是 NONE 才轮到 g 生效
F composition(F f, F g) {
    return f == NONE ? g : f;
}
F id() { return NONE; }
```

区间赋值 + 区间和要把两者拼起来：S 存 {sum, len}, mapping 里 f != NONE 时返回 {f * x.len, x.len}。同理「区间赋值 + 区间加」的复合标记要写成 struct F { ll assign, add; }, composition 按「先 g 再 f」的顺序把两层压平——这类复合标记的分配律要在纸上验一遍再敲。